

Genetic Algorithm with Multiple Fitness Functions for Generating Adversarial Examples

Chenwang Wu*, Wenjian Luo[†], Nan Zhou*, Peilan Xu*, and Tao Zhu[‡]

* School of Computer Science and Technology, University of Science and Technology of China, Hefei 230027, Anhui, China
{wcw1996, hinanmu, xpl}@mail.ustc.edu.cn

[†]School of Computer Science and Technology, Harbin Institute of Technology, Shenzhen 518000, Guangdong, China
luowenjian@hit.edu.cn

[‡]School of Software Engineering, University of South China, Hengyang 421001, Hunan, China
tzhu@usc.edu.cn

Abstract—Studies have shown that deep neural networks (DNNs) are susceptible to adversarial attacks, which can cause misclassification. The adversarial attack problem can be regarded as an optimization problem, then the genetic algorithm (GA) that is problem-independent can naturally be designed to solve the optimization problem to generate effective adversarial examples. Considering the dimensionality curse in the image processing field, traditional genetic algorithms in high-dimensional problems often fall into local optima. Therefore, we propose a GA with multiple fitness functions (MF-GA). Specifically, we divide the evolution process into three stages, i.e., exploration stage, exploitation stage, and stable stage. Besides, different fitness functions are used for different stages, which could help the GA to jump away from the local optimum.

Experiments are conducted on three datasets, and four classic algorithms as well as the basic GA are adopted for comparisons. Experimental results demonstrate that MF-GA is an effective black-box attack method. Furthermore, although MF-GA is a black-box attack method, experimental results demonstrate the performance of MF-GA under the black-box environments is competitive when comparing to four classic algorithms under the white-box attack environments. This shows that evolutionary algorithms have great potential in adversarial attacks.

I. INTRODUCTION

In recent years, deep learning has played a significant role in various intelligent fields [1]–[3], given its strong feature learning ability. However, recent studies have shown that deep neural networks (DNNs) are extremely vulnerable [4]; that is, regardless of the classification accuracy, an attacker’s carefully modified data that cannot be detected by the human eye but can be misclassified by the networks. Such a malicious DNN model attack is called an adversarial attack, and the malicious samples generated by the attack are called an adversarial example. Due to the irreplaceable role of deep learning in many fields, it is very important to study various adversarial attacks to explain such perturbations’ existence and improve the models’ robustness.

This study is supported by the National Key Research and Development Program of China (No. 2020YFB2104003) and the National Natural Science Foundation of China (No. 61573327). (Corresponding author: Wenjian Luo.)

According to the knowledge obtained, adversarial attacks can be classified into white-box attacks and black-box attacks [5]. As the name implies, a white-box is where all information is transparent to the attacker, including weights, datasets, model structures, and even training details. Another type of attack is the black-box attack. Such an attack treats the model as a black box and can only attack it based on the input data and the corresponding output. Under black-box attacks with limited information, it is difficult to achieve performance similar to white-box attacks. However, in practical applications, attackers often know little about the model information; thus, the black-box attacks are more practical. Currently, there are some studies using evolutionary algorithms to generate adversarial samples [6], [7]. Unlike traditional calculus-based optimization methods that use internal information (such as gradients), the evolutionary algorithm does not require the model’s internal information by its special search strategy. Therefore, attack techniques based on evolutionary algorithms are often grouped into the black-box attack methods.

In this paper, a genetic algorithm with multiple fitness functions (MF-GA) is proposed to generate adversarial examples. Although some recent studies about adversarial examples [6]–[9] are based on evolutionary algorithms, but our algorithm is different from theirs. First of all, AdversarialPSO [8] and PSOAttack [9] adopted particle swarm optimization, while we use the genetic algorithm. Secondly, GenAttack [6] and POBA-GA [7] are only evolving partial pixels due to the dimensionality curse, while our approach is to simultaneously evolve all pixels because it is good practice for the algorithm to decide which pixels are used to attack automatically. Although this is more challenging for thousands of pixels, it is more in line with general perturbations. To this end, our algorithm uses multiple strategies such as multi-fitness functions and discrete encoding to avoid falling into local optima and reduce the perturbation search space, which is also different from existing work.

The problem of adversarial attacks can be abstracted into different optimization problems. If the optimization algorithm is powerful enough, it is possible to find the adversarial perturbations by optimizing any of these optimization problems. However, owing to the increase in dimensions, it becomes easy

to fall into a local optimum by considering only a single fitness function such as in traditional GA. For this reason, MF-GA is proposed in which different fitness functions are used in different optimization stages. Although it cannot guarantee that other fitness values will evolve in a good direction similar to the fitness currently in use, the evolutionary jitter of MF-GA means that the algorithm has more potential to jump out of the local optimum and find high-quality solutions that have never been evolved before.

In summary, the contributions of the paper are as follows:

- 1) A GA based on multiple fitness functions (MF-GA) is proposed. Different fitness functions in different evolutionary stages could mitigate falling into the local optimal solution. Existing GA-based methods use partial pixel evolution or block optimization strategies to avoid the disaster of dimensionality. The proposed MF-GA uses the discrete encoding and relaxes the attack objective to evolve all pixels that are more in line with general perturbations, which provides new ideas for subsequent adversarial attacks based on evolutionary algorithms.
- 2) Experiments on three datasets are done in this paper, and four baselines, i.e., FGSM [10], BIM [11], MI-FGSM [12], and SPSA [13], as well as a basic GA, are adopted for comparisons. Experimental results indicate that the proposed MF-GA could achieve the competitive performance under black-box environments. Furthermore, its performance under black-box environments is competitive when comparing with four baselines in the white-box scenario.

The rest of this paper is organized as follows. Section II introduces some related work. Section III defines the optimization problem. Section IV describes the proposed algorithm. Section V reports the experimental results and the corresponding analysis. Section VI briefly concludes the work in this paper.

II. RELATED WORK

Since Szegedy et al. [4] proposed the concept of adversarial examples, multiple studies have shown that DNNs are susceptible to small perturbations [10], [11], [14]. Goodfellow et al. [10] proposed the fast gradient sign method (FGSM), which adds a small perturbation to the target model in the gradient direction. Subsequent iterations of FGSM (BIM) [11] and momentum-based iterative FGSM (MI-FGSM) [12] further improved the attack performance. Moosavi-Dezfooli et al. [15] generated adversarial samples by computing the classifier's decision boundaries and adding perturbations towards the decision boundaries. Associating the minimum perturbation and loss function, Carlini et al. [14] proposed three optimization-based attack methods that limit the perturbation norm of L_0 , L_2 , and L_∞ , respectively. Based on the adversarial noise's transferability, Papernot et al. [16] proposed a black-box attack, which trains a substitute model by constructing new data and attacks the target model by disturbing the substitute model. Uesato et al. [13] used the gradient-free optimization SPSA to perform an adversarial attack, which is more effective when

the model is not differentiable. Huang et al. [17] proposed an intermediate level attack (ILA) to address the intermediate feature map to improve the perturbations' transferability. Chen et al. [18] proposed a zeroth-order optimization based black-box attack. Based on [18], AutoZoom [19] further improved query efficiency. Instead of adding perturbations to the original image, Song et al. [20] modeled the data distribution through an auxiliary generative model and looked for adversarial samples that could be misclassified in the distribution potential space.

By limiting the L_0 norm of the perturbation, Su et al. [21] used differential evolution to modify one pixel to attack different DNN models successfully. Inspired by the one-pixel attack, other evolutionary algorithms for adversarial attacks have been developed. Based on [21], Lin et al. [22] further proposed BMI-FGSM using differential evolution. As a kind of evolutionary algorithms, genetic algorithms are naturally used to generate adversarial examples [6], [7]. However, stuck in the curse of dimensionality, these methods only evolve part of the pixels. As a non-gradient evolutionary search algorithm, particle swarm optimization has the special black box adaptability, and it has also been studied for black-box attacks [8], [9]. Suzuki et al. [23] proposed various multi-objective optimization problems about adversarial attacks and used the MOEA/D [24] to solve the problems to generate the adversarial samples. CMA-ES [25] is considered an advanced algorithm in evolutionary computing, and it is also used to generate adversarial samples [26]. It is worth noting that these evolution-based methods do not need to obtain the model's internal parameters and thus are black-box attacks.

In addition to image classification [27], increasingly more research shows that adversarial perturbations also exist in the fields of semantic segmentation [28], speech recognition [29], [30], visual question answering [31], reinforcement learning [32], etc.

With the introduction of numerous attack methods, many studies have attempted to defend against adversarial attacks. Adding adversarial samples to the training set can learn the specific characteristics of the adversarial examples and effectively improve the model's robustness [10], [33], [34]. Papernot et al. [35] proposed using distillation networks to hide gradients to resist multiple types of adversarial attacks. In addition to improving the model's robustness, image denoising is also a defensive strategy. The high-level representation guided denoiser (HGD) [36] and the Defense-GAN [37] are essentially denoisers that attempt to remove the noise from the adversarial samples. Image compression [38], [39] has also been shown to reduce the damage caused by the perturbation. Besides, it is equally important to detect adversarial samples as a means of protecting the classifier. It has been proved that it is feasible to train a helper model to determine whether the sample is contaminated [40]. Xu et al. [41] used feature squeezing to determine the consistency of the predictions before and after processing. Also, some work [42]–[44] has been performed to identify adversarial perturbations by analyzing the samples' internal statistical features.

III. PROBLEM DEFINITION

The adversarial attack's goal is to generate adversarial examples that are difficult for the human eye to perceive but can cause the classifier to make incorrect predictions. Thus, in most cases, a practical attack must satisfy two guidelines: (1) the smaller the added perturbation, the better, and (2) the classifier must misclassify and maximizes the difference in confidence between the incorrect and correct categories. Therefore, the problem of finding an adversarial perturbation can be transformed into an optimization problem.

Suppose the target image is x , the added noise is η , and $f(x)$ denotes the prediction vector of the model for input x , and $f_y(x)$ indicates the confidence of classification for category y . Then, the calculation of the noise η could be transformed into the following constrained optimization problem:

$$\begin{aligned} \arg \min_{\eta} & (\lambda \cdot \eta + (1 - \lambda) \cdot (f_l(x + \eta) - f_{adv}(x + \eta))) \\ \text{s.t.} & f_{adv}(x + \eta) - f_l(x + \eta) > 0. \end{aligned} \quad (1)$$

Where λ is the balance factor used to balance the influence of the perturbation size and attack performance, l is the correct label, and adv indicates the error label that the attacker wants the classifier to predict. The constraints here can guarantee that the classification of the classifier is incorrect.

Existing attack methods generally measure the perturbation size by calculating the L_0 , L_2 , and L_∞ norms of the perturbation. Regardless of which measurement method is used, the perturbation noise η can be expressed as a vector of the same dimension as the target image x . For example, in the MNIST dataset, the most commonly used dataset in the image classification field, the image size of 28×28 is equivalent to the dimension of the perturbation η of up to 784, and the slightly larger $32 \times 32 \times 3$ CIFAR-10 images are up to 3072 dimensions. Therefore, generating adversarial examples can be regarded as solving a large-scale optimization constrained problem in a sense. In addition, the high degree of non-linearity of DNN results in interactions among pixels; thus, it is also an additively non-separable problem. The interactions among pixels make it difficult for existing large-scale optimization algorithms based on divide and conquer to play their roles.

For the constrained optimization problem shown in (1), we must consider both the perturbation size and the confidence of the category. First, the value of λ in solving the problem, undoubtedly adds difficulty to the already tricky high-dimensional optimization. Besides, considering that the pixel values of the normalized image x fall in the range $[0, 1]$, the range of η is also a floating point number of $[0, 1]$, and the vast search space also makes the high-dimensional optimization problem more difficult.

Therefore, in this paper, we set the perturbation η by a way inspired by FGSM [10]. In FGSM,

$$\eta = \epsilon \cdot \text{sign}(\nabla_x J(x, f(x))),$$

where $J(x, f(x))$ is the loss of the model. ϵ is the perturbation step size, which takes on a small value. We believe that the

perturbation noise within the step range is difficult to perceive by the human eye; that is, the added noise is meaningful.

This paper uses the perturbation η as follow.

$$\eta = \epsilon \cdot \mathcal{D}. \quad (2)$$

The perturbation noise used is equivalent to that used by FGSM. However, the difference here is that we do not get the value of $\nabla_x J(x, f(x))$. We try to find a discrete vector $\mathcal{D} \in \{-1, 0, 1\}^m$, where m is the number of pixels in the images. At this point, the search space consists of an m -dimensional discrete space with each dimension of three integer values. Moreover, ϵ is specified in advance, so the first half of (1) can be ignored.

Finally, we further relax the target problem and remove the constraints in (1). The starting point of the problem is to attack the target sample thoroughly; however, the attack's success is not assured. In combination with the perturbations shown in (2), this paper proposed the following optimization problem for generating adversarial examples:

$$\arg \min_{\mathcal{D}} (f_l(x + \epsilon \cdot \mathcal{D}) - f_{adv}(x + \epsilon \cdot \mathcal{D})). \quad (3)$$

By minimizing (3), we can maximize the difference between the confidence of the incorrect category and the correct category. If the value of $f_l(x + \epsilon \cdot \mathcal{D}) - f_{adv}(x + \epsilon \cdot \mathcal{D})$ is less than 0, the predicted probability of incorrect category will be greater than the correct category; that is, the attack is successful. In summary, (3) is an appropriate optimization problem for generating adversarial examples.

IV. ALGORITHM DESCRIPTION

In this section, the proposed genetic algorithm based on multi fitness function (MF-GA) will be described, which is used to generate adversarial samples.

GAs have been widely used to solve complex optimization problems. Like traditional GA, MF-GA also includes a series of processes such as selection, crossover, and mutation. The details of MF-GA are given as follows.

Encoding: The premise that GA works well is that it requires good encoding. This paper uses a discrete value encoding. Unlike the discrete coding commonly using binary encoding $\{0, 1\}$, we use the ternary vector $\{-1, 0, 1\}^m$ to represent a candidate solution (chromosome), where m is the dimension of the image. It can be seen from (2) that the perturbation can be obtained by multiplying the candidate solution by a preset perturbation step ϵ .

Initialization: For the generation of initial individuals, a better setting is to determine the optimal solution's distribution characteristics and generate initial candidate solutions within the distribution to accelerate the convergence. Unfortunately, it is difficult to form a unified theoretical interpretation of the perturbation, not to mention the adversarial noise's distribution characteristics. To ensure that the generated initial solution uniformly covers the entire search space, we use the uniform random initialization:

$$P_{i,j}^0 = \text{rand_choice}(\{-1, 0, 1\}).$$

Where $P_{i,j}^0$ represents the j -th position of the i -th individual of the initial population P^0 .

Selection Operator: For selecting individuals used in crossover and mutation operations, we use tournament selection. Based on the fact that individuals with poor fitness values are not necessarily difficult to attack the DNN model, we choose a binary tournament with low selection pressure to maintain population diversity. We select two individuals from the population each time without putting them back, and then pick the better one, who will act as one of the parent individuals of the subsequent crossover. If all the individuals in the population have been traversed, all the individuals are returned, and the operation is repeated until it is enough to produce a population with the same size of the parent population.

Crossover Operator: Crossover, which generates new individuals by recombining the genes of individuals in the population, is the key to generating offspring. We choose a uniform crossover operator in which each bit of the offspring is selected with equal probability from parent individuals. Here each crossover operation uses two individuals selected through the proposed tournament, and each bit is chosen with a probability of 0.5 from a parent individual:

$$P_{i,j}^t = \begin{cases} P_{s_{i1},j}^{t-1} & \text{rand}(0,1) < 0.5 \\ P_{s_{i2},j}^{t-1} & \text{otherwise} \end{cases}.$$

Where $P_{i,j}^t$ means the j -th position of the i -th individual in the t -th generation population P^t , s_{i1} and s_{i2} respectively represent the indexes of two parent individuals, and $\text{rand}(0,1)$ is a uniform random number between 0 and 1.

Mutation Operator: Mutation, which produces new individuals by modifying certain bits of the individuals in the population, makes the algorithm more capable of local random search. The mutation operator is explained as follows.

$$P_{i,j}^t = \begin{cases} \text{rand}(\{-1, 0, 1\} - \{P_{i,j}^t\}) & \text{rand}(0,1) < \frac{1}{m} \\ P_{i,j}^t & \text{otherwise} \end{cases}.$$

Where m is the dimension of each individual. This means that the averagely one bit per individual will be changed. In the above formula, $\text{rand}(\{-1, 0, 1\} - \{P_{i,j}^t\})$ randomly choose one from $\{-1, 0, 1\}$ after excluding $P_{i,j}^t$.

Fitness Function: In evolutionary algorithms, the selection of good individuals depends on the fitness function. The fitness function is related to the optimization problem, which is shown in (3) in Section III. Here, adv chooses the error label with the highest confidence, which is represented by l^- . Thus, the optimization problem can be re-written as follow.

$$F = f_l(x + \epsilon \cdot \mathcal{D}) - f_{l^-}(x + \epsilon \cdot \mathcal{D}).$$

The smaller the value F is, the better the individual, the more likely it is to survive.

In the ideal case, the optimization problem mentioned above can be optimized to facilitate an attack. However, the dimensional curse makes it very challenging for existing GAs.

Therefore, we lead to the core idea of this paper, using multiple fitness functions for different evolutionary stages.

Specifically, the paper uses four fitness functions:

- Reduce the confidence of the correct category:

$$F_1 = f_l(x + \eta).$$

- Increase the confidence of the error category with maximum confidence:

$$F_2 = -f_{l^-}(x + \eta).$$

- Increase the difference in the confidence between the error category with maximum confidence and the correct category:

$$F_3 = f_l(x + \eta) - f_{l^-}(x + \eta).$$

- Maximize the confidence of the least likely category:

$$F_4 = -f_{least}(x + \eta).$$

Here, we choose the category with the lowest predicted probability for the least likely category, i.e., $least = \arg \min_i f_i(x)$. Inspired by [11], the purpose of introducing F_4 is to influence the prediction of all categories by perturbing the least likely category $least$ and then explore the most likely attack category l^- .

The specific MF-GA is shown in Alg. 1. Line 2 generates a set of candidate solutions using uniform random initialization, and line 3 uses the proposed fitness function to evaluate the initial individuals' fitness values. Line 5 through line 21 evolve the population for T generations. Specifically, each generation of evolution includes the use of binary tournaments to select the parent, uniform crossover, uniform mutation, and the selection of the best N individuals from the parent and child individuals according to the fitness values. Line 22 through line 23 generates final adversarial examples.

In Alg. 1, $\text{argsort}(\text{fit_list}^{t+1})$ is a function that sorts the individuals in descending order according to the fitness values, and $\text{fit_list}[i]$ represents the fitness value under F_i . It's worth noting that although the number of evaluations of F_1 to F_4 seems to require four times for each individual, after calculating $f(x + \eta)$, we can quickly obtain four fitness values, and the cost can be ignored when comparing to calculating $f(x + \eta)$.

The core of Alg. 1 is the multiple fitness evaluation of lines 6-12. In MF-GA, we divide the optimization process into three phases: the exploration stage (lines 6-7), the exploitation stage (lines 8-9), and the stable stage (lines 10-12).

For a well-trained network, except for the correct category, other categories' confidence in the initial stage is very low. At this time, the error category with the highest confidence is not necessarily the most likely to be attacked; thus, in the beginning, we randomly select F_1 and F_4 that do not contain the victim category and explore the category that is most likely to attack.

Algorithm 1 MF-GA

Input: Target image x , image dimensions m , population size N , evolutionary generations T , population $P = \emptyset$, perturbation threshold ϵ , stage thresholds $\Delta_1, \Delta_2, p_1, p_2$.

Output: Adversarial example x^* .

```

1:  $fit\_origin = Evaluate\_MF(x, \mathbf{0});$ 
2:  $P^0 = Initialize(N, m);$ 
3:  $fit\_list^0 = Evaluate\_MF(x, P^0);$ 
4:  $t = 0;$ 
5: while  $t < T$  and  $all(fit\_list^t[3] >= 0)$  do
6:   if  $any(fit\_origin[2] - fit\_list^t[2] < \Delta_1)$  and
      $t < T * p_1$  then
7:      $i = random\_choice(\{1, 4\});$ 
8:   else if  $any(fit\_list^t[3]) > \Delta_2$  and
      $t < T * p_2$  then
9:      $i = random\_choice(\{1, 2, 3\});$ 
10:  else
11:     $i = roulette\_selection(\varphi);$ 
12:  end if
13:   $parents = Selection(P^t, fit\_list^t[i]);$ 
14:   $P^{t+1} = Crossover(parents);$ 
15:   $P^{t+1} = Mutation(P^{t+1});$ 
16:   $P^{t+1} = P^{t+1} \cup P^t;$ 
17:   $fit\_list^{t+1} = fit\_list^t \cup Evaluate\_MF(x, P^{t+1});$ 
18:   $P^{t+1} = P^{t+1}[argsort(fit\_list^{t+1}[i])[1:N];$ 
19:   $fit\_list^{t+1} = fit\_list^{t+1}[argsort(fit\_list^{t+1})[1:N];$ 
20:   $t = t + 1;$ 
21: end while
22:  $\mathcal{D}$  = the best individual in  $P^t$  according to  $fit\_list^t[3];$ 
23:  $x^* = x + \epsilon \cdot \mathcal{D};$ 
24: return  $x^*$ 

```

When the confidence increment of category l^- is greater than a threshold Δ_1 or the number of generations reaches a threshold $T * p_1$, the algorithm enters the second stage. In the exploitation state, we desire to find a variety of high-quality solutions through multiple fitness functions so that each generation chooses a fitness function from F_1, F_2 , and F_3 in a random manner. However, each generation does not guarantee that all individuals' adaptive values have evolved in a good direction; this jitter phenomenon allows us to introduce a stable phase.

When the attack is about to succeed ($fit_list[3] < \Delta_2$) or the algorithm is about to end ($t > T * p_2$), it enters a stable phase. At this stage, the more direct F_3 was mainly used, and F_1 and F_2 were selected with a lower probability. We use the roulette algorithm to solve this uneven probability selection, which corresponds to the function $roulette_selection(\varphi)$ on line 11 of Alg.1. The specific probability φ of each fitness in the experiments will be introduced in section V-B.

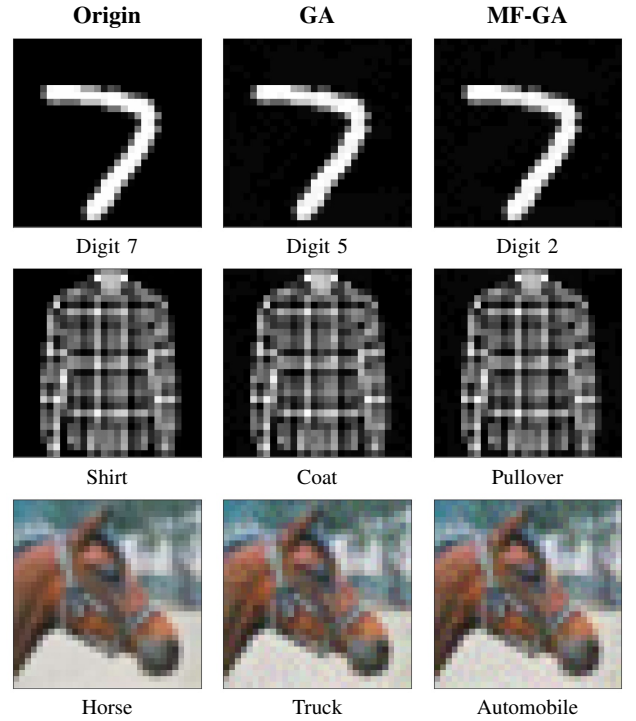


Fig. 1. Some examples of adversarial attacks. The first column is the original images, and the next two columns represent the adversarial images generated by the two attack methods used in the paper. Below each image represents the category predicted by the classifier. The perturbation step ϵ is set to 0.03.

Algorithm 2 GA

Input: Target image x , image dimensions m , population size N , maximum evolutionary generations T , population $P = \emptyset$, perturbation threshold ϵ .

Output: Adversarial example x^* .

```

1:  $P^0 = Initialize(N, m);$ 
2:  $fit^0 = Evaluate(x, P^0);$ 
3:  $t = 0;$ 
4: while  $t < T$  and  $all(fit^t >= 0)$  do
5:    $parents = Selection(P^t, fit^t);$ 
6:    $P^{t+1} = Crossover(parents);$ 
7:    $P^{t+1} = Mutation(P^{t+1});$ 
8:    $fit^{t+1} = fit^t \cup Evaluate(x, P^{t+1});$ 
9:    $P^{t+1} = P^{t+1} \cup P^t;$ 
10:   $P^{t+1} = P^{t+1}[argsort(fit^{t+1})[1:N];$ 
11:   $fit^{t+1} = fit^{t+1}[argsort(fit^{t+1})[1:N];$ 
12:   $t = t + 1;$ 
13: end while
14:  $\mathcal{D}$  is set to the best individual;
15:  $x^* = x + \epsilon \cdot \mathcal{D};$ 
16: return  $x^*$ 

```

V. EXPERIMENTS

A. Experimental Settings

1) *Datasets and models:* We choose three widely used public datasets, MNIST, Fashion-MNIST (FMNIST), and CIFAR-10, for evaluations. MNIST and FMNIST contain

gray-scale images with a resolution of 28×28 , and CIFAR-10 contains color images with a resolution of $32 \times 32 \times 3$. This shows that the dimensionality of the attack problem for MNIST and FMNIST is 784, and the dimensionality for CIFAR-10 is 3072. The optimization of such a large-scale adversarial problem is undoubtedly challenging.

The selection of the classifier model is based on the advanced performance of the classification field. MNIST uses a simple 7-layer CNN structure, FMNIST uses an 8-layer CNN structure, and CIFAR-10 uses the VGG-16 model. The learning rate is set to 0.001, and data augmentation operations, such as rotation and horizontal flipping, are used, all data are normalized to $[0, 1]$, and each dataset is trained for 100 epochs. At the end of the training, the models on MNIST, FMNIST, and CIFAR-10 achieve the classification accuracy of 99.2%, 94.2%, and 92.2% in the test set, respectively. It is worth noting that when testing adversarial attacks, we remove the misclassified images of the model and only consider attacking images that can be correctly classified.

2) *General Settings*: We select the first 1000 samples from each dataset for testing, which is equivalent to a total of 3,000 minimization optimization problems. We perform 15 independent repeated experiments on each image. The number of individuals in the population is set to 50. For a fair comparison, each algorithm uses the same maximum evolutionary generation of 500 generations, representing 25,000 fitness evaluations.

Besides, the algorithms proposed in the paper all limit the maximum value of the perturbation (L_∞ norm); thus, we choose FGSM [10], BIM [11], MI-FGSM [12], and SPSA [13], which also limit the L_∞ norm of the perturbation, for the performance comparison. Among them, FGSM adds noise in the direction of the gradient sign, BIM and MI-FGSM respectively denote the iterative version of FGSM whether to consider momentum, and SPSA uses the SPSA optimization algorithm to maximize the attack risk. In addition, we also adopt the GA without multiple fitness functions for comparison, where F_3 is adopted as the fitness function. The specific algorithm of GA is shown in Alg. 2. The perturbation step ϵ used by the algorithm is 0.03 or 0.05. In Fig. 1, we list some adversarial samples generated by the attacks proposed in the paper. The generated adversarial examples are difficult to distinguish from the original image; however, the classifier misclassified them.

For reproducibility, the source codes of this paper are available at <https://github.com/Daftstone/GA-based-Attack>.

B. Effectiveness of the Multiply Fitness Functions

Here, we compare GA and MF-GA to evaluate the effectiveness of multiple fitness functions. For MF-GA, Δ_1 is 0.25, p_1 is 0.5, Δ_2 is 0.03, p_2 is 0.9, and $\varphi = (0.25, 0.25, 0.5)$ for F_1 , F_2 and F_3 , respectively. The attack success rate is listed in Table I. The average number of evaluations on each image is shown in Table II, which lists the number of fitness evaluations for successful samples (*success*) and all samples (*all*).

First of all, it is easy to see from Table I that MF-GA is significantly better than GA. When ϵ is 0.03, the average performance of FMNIST dataset is improved by more than 7%, which is a surprising result. Second, from the perspective of the number of fitness evaluations in Table II, an increase can be seen. This paper proposes multiple fitness functions to avoid the algorithm falling into a local optimum, and some evolutionary generations in the attack process will produce individuals that are inferior to their parents. Therefore, it is reasonable that the number of fitness evaluations of MF-GA will increase to a certain extent.

C. Comparison with baselines

In this section, we compare GA and MF-GA with some classical methods. To unify the standards for the perturbation and make a fair comparison, we choose four typical attacks that also limit the L_∞ norm of the perturbation: FGSM, BIM, MI-FGSM, and SPSA. For GA and MF-GA, their attack success rates are the average results of 15 independent experiments.

Our method only needs the model's feedback information, so the attack is executed in the black box. On the contrary, the baseline method needs to use gradient information. For the sake of comparison, we do experiments about baselines under both white-box attack and black-box attack. For black-box attacks, the target model cannot be directly processed. To this end, we use a local substitute model to generate adversarial examples, which is inspired by [16]. At this time, the alternative model is visible to the attacker, so we transform the black box attack into the white box attack. Specifically, MNIST uses a simple 8-layers CNN structure, FMNIST uses a 7-layer CNN structure, and CIFAR-10 uses the ResNet18 model, which are different from the target models described in section V-A1.

The current mainstream black-box attack methods utilize the transferability of perturbations to generate adversarial samples through substitute models. However, the transferability of perturbations is limited. From Table III, it can be seen that the black-box performance of the first three classic methods, such as FGSM, has significantly decreased. SPSA attack is a gradient-free black-box attack method, so its performance has not changed. Comparing to these four baselines, MF-GA is the best under the black-box environments.

Moreover, as shown in Table III, even comparing to the performance of four baselines under white-box environments, the performance of MF-GA under black-box environments is competitive, which is a very gratifying result. These results show that the use of evolutionary algorithms for black-box attacks is very promising.

VI. CONCLUSION

In this paper, we propose a genetic algorithm based on multiple fitness functions, i.e., MF-GA, for generating adversarial samples. We analyze the adversarial attack and divide the evolution process into three stages. Different stages use different fitness functions, which will cause evolutionary jitter

TABLE I
SUCCESS RATES OF ATTACK BASED ON GA AND MF-GA

Dataset	Algorithm	$\epsilon=0.03$					$\epsilon=0.05$				
		Best	Worst	Median	Mean	StDev	Best	Worst	Median	Mean	StDev
MNIST	GA	0.335	0.324	0.332	0.332	0.0028	0.777	0.767	0.770	0.771	0.0032
	MF-GA	0.360	0.351	0.356	0.356	0.0024	0.797	0.788	0.792	0.792	0.0022
FMNIST	GA	0.921	0.909	0.916	0.916	0.0038	0.972	0.967	0.968	0.969	0.0014
	MF-GA	0.990	0.985	0.987	0.987	0.0013	1	1	1	1	0
CIFAR-10	GA	1	0.999	0.999	0.999	0.0005	1	1	1	1	0
	MF-GA	1	0.999	1	1	0.0005	1	1	1	1	0

TABLE II
FITNESS EVALUATIONS OF ATTACKS BASED ON GA AND MF-GA

Dataset	Algorithm	$\epsilon=0.03$						$\epsilon=0.05$					
		Mean		Median		StDev		Mean		Median		StDev	
		success	all	success	all	success	all	success	all	success	all	success	all
MNIST	GA	5728.5	18608.3	5749.9	18609.8	91.37	59.22	3255.4	8229.2	3257.4	8227.0	61.69	56.39
	MF-GA	6917.2	18559.1	6927.0	18556.1	137.48	36.45	4759.5	8972.3	4765.5	8973.7	66.64	36.73
FMNIST	GA	1210.9	3218.8	1206.3	3197.2	48.19	57.60	460.2	1230.8	461.5	1239.0	22.55	27.95
	MF-GA	2088.0	2378.3	2093.9	2377.3	37.57	26.14	542.8	568.9	540.1	567.2	6.17	6.54
CIFAR-10	GA	988.9	1001.7	990.1	997.8	13.95	12.61	379.8	379.1	379.2	378.0	2.77	2.97
	MF-GA	1100.6	1110.2	1100.8	1108.8	17.38	16.27	405.9	403.2	405.8	405.2	4.02	4.31

TABLE III
ATTACK SUCCESS RATE COMPARISON WITH BASELINES

Knowledge	Attack	MNIST		FMNIST		CIFAR-10	
		$\epsilon=0.03$	$\epsilon=0.05$	$\epsilon=0.03$	$\epsilon=0.05$	$\epsilon=0.03$	$\epsilon=0.05$
White-box	FGSM	0.051	0.173	0.437	0.556	0.767	0.857
	BIM	0.300	0.56	0.846	0.973	0.996	1
	MI-FSGM	0.334	0.748	0.958	0.992	0.999	0.999
	SPSA	0.274	0.734	0.959	1	1	1
Black-box	FGSM	0.008	0.019	0.199	0.323	0.218	0.395
	BIM	0.018	0.152	0.285	0.476	0.427	0.691
	MI-FSGM	0.015	0.145	0.384	0.595	0.616	0.836
	SPSA	0.274	0.734	0.959	1	1	1
	GA	0.332	0.771	0.916	0.969	0.999	1
	MF-GA	0.356	0.792	0.987	1	1	1

and then jump out of the local optimal areas. Although the attack problem seems to be a high-dimensional optimization problem, experimental results indicate that the performance of MF-GA is competitive.

Moreover, MF-GA, which is a black-box attack, can achieve competitive attack performance when comparing to four white-box attacks. Therefore, research on adversarial attacks based on highly robust evolutionary algorithms may overcome the shortcomings of the current black-box attack's insufficient performance. However, it is undeniable that adversarial attacks based on evolutionary algorithms require many access queries to the target model. How to reduce the number of queries of the algorithm will require a further study.

REFERENCES

- [1] G. Hinton, L. Deng, D. Yu, G. Dahl, A.-r. Mohamed, N. Jaitly, A. Senior, V. Vanhoucke, P. Nguyen, B. Kingsbury *et al.*, "Deep neural networks for acoustic modeling in speech recognition," *IEEE Signal Processing Magazine*, vol. 29, 2012.
- [2] Y. Sun, Y. Chen, X. Wang, and X. Tang, "Deep learning face representation by joint identification-verification," in *Advances in Neural Information Processing Systems*, 2014, pp. 1988–1996.
- [3] K. Cho, B. van Merriënboer, Ç. Gülçehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio, "Learning phrase representations using rnn encoder-decoder for statistical machine translation," in *EMNLP*, 2014.
- [4] C. Szegedy, W. Zaremba, I. Sutskever, J. Bruna, D. Erhan, I. Goodfellow, and R. Fergus, "Intriguing properties of neural networks," in *2nd International Conference on Learning Representations, ICLR 2014*, 2014.
- [5] N. Akhtar and A. Mian, "Threat of adversarial attacks on deep learning in computer vision: A survey," *IEEE Access*, vol. 6, pp. 14410–14430, 2018.
- [6] M. Alzantot, Y. Sharma, S. Chakraborty, H. Zhang, C.-J. Hsieh, and M. B. Srivastava, "Genattack: Practical black-box attacks with gradient-free optimization," in *Proceedings of the Genetic and Evolutionary Computation Conference*, 2019, pp. 1111–1119.
- [7] J. Chen, M. Su, S. Shen, H. Xiong, and H. Zheng, "Poba-ga: Perturbation optimized black-box adversarial attacks via genetic algorithm," *Computers & Security*, vol. 85, pp. 89–106, 2019.

- [8] R. Mosli, M. Wright, B. Yuan, and Y. Pan, "They might not be giants crafting black-box adversarial examples using particle swarm optimization," in *European Symposium on Research in Computer Security*. Springer, 2020, pp. 439–459.
- [9] Z. Yi, S. Li, J. Yu, Y. Tan, and Q. Wu, "A novel misclassification attack against black box neural network classifiers," in *2018 14th International Conference on Semantics, Knowledge and Grids (SKG)*. IEEE, 2018, pp. 38–45.
- [10] I. J. Goodfellow, J. Shlens, and C. Szegedy, "Explaining and harnessing adversarial examples," *arXiv preprint arXiv:1412.6572*, 2014.
- [11] A. Kurakin, I. Goodfellow, and S. Bengio, "Adversarial examples in the physical world," *arXiv preprint arXiv:1607.02533*, 2016.
- [12] Y. Dong, F. Liao, T. Pang, H. Su, J. Zhu, X. Hu, and J. Li, "Boosting adversarial attacks with momentum," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2018, pp. 9185–9193.
- [13] J. Uesato, B. O'donoghue, P. Kohli, and A. Oord, "Adversarial risk and the dangers of evaluating against weak attacks," in *International Conference on Machine Learning*. PMLR, 2018, pp. 5025–5034.
- [14] N. Carlini and D. Wagner, "Towards evaluating the robustness of neural networks," in *2017 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2017, pp. 39–57.
- [15] S.-M. Moosavi-Dezfooli, A. Fawzi, and P. Frossard, "Deepfool: a simple and accurate method to fool deep neural networks," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2016, pp. 2574–2582.
- [16] N. Papernot, P. McDaniel, I. Goodfellow, S. Jha, Z. B. Celik, and A. Swami, "Practical black-box attacks against machine learning," in *Proceedings of the 2017 ACM on Asia Conference on Computer and Communications Security*. ACM, 2017, pp. 506–519.
- [17] Q. Huang, I. Katsman, H. He, Z. Gu, S. Belongie, and S.-N. Lim, "Enhancing adversarial example transferability with an intermediate level attack," in *Proceedings of the IEEE International Conference on Computer Vision*, 2019, pp. 4733–4742.
- [18] P.-Y. Chen, H. Zhang, Y. Sharma, J. Yi, and C.-J. Hsieh, "Zoo: Zeroth order optimization based black-box attacks to deep neural networks without training substitute models," in *Proceedings of the 10th ACM Workshop on Artificial Intelligence and Security*, 2017, pp. 15–26.
- [19] C.-C. Tu, P. Ting, P.-Y. Chen, S. Liu, H. Zhang, J. Yi, C.-J. Hsieh, and S.-M. Cheng, "Autozoom: Autoencoder-based zeroth order optimization method for attacking black-box neural networks," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 33, 2019, pp. 742–749.
- [20] Y. Song, R. Shu, N. Kushman, and S. Ermon, "Constructing unrestricted adversarial examples with generative models," in *Advances in Neural Information Processing Systems*, 2018, pp. 8312–8323.
- [21] J. Su, D. V. Vargas, and K. Sakurai, "One pixel attack for fooling deep neural networks," *IEEE Transactions on Evolutionary Computation*, vol. 23, no. 5, pp. 828–841, 2019.
- [22] J. Lin, L. Xu, Y. Liu, and X. Zhang, "Black-box adversarial sample generation based on differential evolution," *Journal of Systems and Software*, vol. 170, p. 110767, 2020.
- [23] T. Suzuki, S. Takeshita, and S. Ono, "Adversarial example generation using evolutionary multi-objective optimization," in *2019 IEEE Congress on Evolutionary Computation (CEC)*. IEEE, 2019, pp. 2136–2144.
- [24] Q. Zhang and H. Li, "Moea/d: A multiobjective evolutionary algorithm based on decomposition," *IEEE Transactions on Evolutionary Computation*, vol. 11, no. 6, pp. 712–731, 2007.
- [25] N. Hansen, "The CMA evolution strategy: A tutorial," *arXiv preprint arXiv:1604.00772*, 2016.
- [26] M. Jere, L. Rossi, B. Hitaj, G. Ciocarlie, G. Boracchi, and F. Koushanfar, "Scratch that! an evolution-based adversarial attack against neural networks," *arXiv preprint arXiv:1912.02316*, 2019.
- [27] N. Zhou, W. Luo, X. Lin, P. Xu, and Z. Zhang, "Generating multi-label adversarial examples by linear programming," in *Proceedings of the 2020 International Joint Conference on Neural Networks (IJCNN 2020)*, 2020, pp. 1–8.
- [28] C. Xie, J. Wang, Z. Zhang, Y. Zhou, L. Xie, and A. Yuille, "Adversarial examples for semantic segmentation and object detection," in *Proceedings of the IEEE International Conference on Computer Vision*, 2017, pp. 1369–1378.
- [29] N. Carlini and D. Wagner, "Audio adversarial examples: Targeted attacks on speech-to-text," in *2018 IEEE Security and Privacy Workshops (SPW)*. IEEE, 2018, pp. 1–7.
- [30] M. Alzantot, B. Balaji, and M. Srivastava, "Did you hear that? adversarial examples against automatic speech recognition," *arXiv preprint arXiv:1801.00554*, 2018.
- [31] X. Zeng, C. Liu, Y.-S. Wang, W. Qiu, L. Xie, Y.-W. Tai, C.-K. Tang, and A. L. Yuille, "Adversarial attacks beyond the image space," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2019, pp. 4302–4311.
- [32] J. Sun, T. Zhang, X. Xie, L. Ma, Y. Zheng, K. Chen, and Y. Liu, "Stealthy and efficient adversarial attacks against deep reinforcement learning," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 34, no. 04, 2020, pp. 5883–5891.
- [33] Y. Wang, D. Zou, J. Yi, J. Bailey, X. Ma, and Q. Gu, "Improving adversarial robustness requires revisiting misclassified examples," in *International Conference on Learning Representations*, 2019.
- [34] A. Madry, A. Makelov, L. Schmidt, D. Tsipras, and A. Vladu, "Towards deep learning models resistant to adversarial attacks," in *International Conference on Learning Representations*, 2018.
- [35] N. Papernot, P. McDaniel, X. Wu, S. Jha, and A. Swami, "Distillation as a defense to adversarial perturbations against deep neural networks," in *2016 IEEE Symposium on Security and Privacy (SP)*. IEEE, 2016, pp. 582–597.
- [36] F. Liao, M. Liang, Y. Dong, T. Pang, X. Hu, and J. Zhu, "Defense against adversarial attacks using high-level representation guided denoiser," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2018, pp. 1778–1787.
- [37] P. Samangouei, M. Kabkab, and R. Chellappa, "Defense-gan: Protecting classifiers against adversarial attacks using generative models," in *International Conference on Learning Representations*, 2018.
- [38] G. K. Dziugaite, Z. Ghahramani, and D. M. Roy, "A study of the effect of jpg compression on adversarial images," *arXiv preprint arXiv:1608.00853*, 2016.
- [39] N. Das, M. Shanbhogue, S.-T. Chen, F. Hohman, S. Li, L. Chen, M. E. Kounavis, and D. H. Chau, "Shield: Fast, practical defense and vaccination for deep learning using jpeg compression," in *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*. ACM, 2018, pp. 196–204.
- [40] J. H. Metzen, T. Genewein, V. Fischer, and B. Bischoff, "On detecting adversarial perturbations," *arXiv preprint arXiv:1702.04267*, 2017.
- [41] C. Xie, Y. Wu, L. v. d. Maaten, A. L. Yuille, and K. He, "Feature denoising for improving adversarial robustness," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2019, pp. 501–509.
- [42] P. Yang, J. Chen, C.-J. Hsieh, J.-L. Wang, and M. I. Jordan, "MI-loo: Detecting adversarial examples with feature attribution," in *AAAI*, 2020, pp. 6639–6647.
- [43] C. Ma, B. Wu, S. Xu, Y. Fan, Y. Zhang, X. Zhang, and Z. Li, "Effective and robust detection of adversarial examples via benford-fourier coefficients," *arXiv preprint arXiv:2005.05552*, 2020.
- [44] J. Martin and C. Elster, "Inspecting adversarial examples using the fisher information," *Neurocomputing*, vol. 382, pp. 80–86, 2020.